

STensor/Tsimplify

Setup

```
In[*]:= << mGRG`STensor`

In[*]:= $PreRead = ReplaceAll[#,
    expr_String => StringReplace[expr, "SP" -> "mGRG`STensor`Private"]] &;

In[*]:= DefKind[Greek, Alphabet["Greek"]]; DefKind[Capital, ToUpperCase /@ Alphabet[]]
```

DnUpPair and UpDnPair

checkOperatorArg

Check the argument of each types of operators for preparing markPairs. If called the markPairs with 'withCheck == False', it should be **explicitly** called this function before the calling the markPairs.

markPairs

Mark pairs of a 'ResetDummied' term according to moveQ and the kind:

```
if moveQ and (dn,up) --> $dndn[uniqSym][kind][idx]
if moveQ and (up,dn) --> $upup[uniqSym][kind][idx]
if !moveQ and (dn,up) --> $dnup[uniqSym][kind][idx]
if !moveQ and (up,dn) --> $updn[uniqSym][kind][idx]
```

```
In[*]:= Tdefine[T, 4]; Tdefine[A, 1]; Tdefine[v, 1]
```

인덱스 쌍이 한 텐서 안에 있는 경우

```
In[*]:= T[ub, lc, ld, lb]
SP`markPairs[List@%, {}, {}]
```

Out[*]=

T_{cda}^a

ToExpression::sntx:
Invalid syntax in or before "10289" [Latin] [ua].
^

ToExpression::sntx:
Invalid syntax in or before "10289" [Latin] [la].
^

Out[*]=

$\{T_{\text{upup[mGRG`STensor`Private`u$10289][Latin][ua]}cd\text{upup[mGRG`STensor`Private`u$10289][Latin][la]}\}$

```
In[*]:= T[ub, lc, ld, lb]
SP`markPairs[List@%, {}, {}] /. $supup[_][k_][a_] → $supup[u][k][a]
```

```
Out[*]=

$$T^a_{cda}$$

```

```
Out[*]=

$$\{T_{\text{\textcolor{brown}{supup}[u][Latin][ua]}cd\text{\textcolor{brown}{supup}[u][Latin][la]}\}$$

```

markPairs 함수의 출력에서 ToExpression::sntx: 경고 메시지를 제거하기 위한 임시 규칙

```
In[*]:= fixPrivateRule = {$f[_][k_][a_] → $f[u][k][a]};
```

```
In[*]:= T[ub, lc, ld, lb]
SP`markPairs[List@%, {}, {}] /. fixPrivateRule
```

```
Out[*]=

$$T^a_{cda}$$

```

```
Out[*]=

$$\{T_{\text{\textcolor{brown}{supup}[u][Latin][ua]}cd\text{\textcolor{brown}{supup}[u][Latin][la]}\}$$

```

연산자가 있는 경우

```
In[*]:= BD[ua, CD[ub, T[la, lb, lc, ud]]]
SP`markPairs[List@%, {}, {}] /. fixPrivateRule
```

```
Out[*]=

$$\partial^a \nabla^b T^d_{abc}$$

```

```
Out[*]=

$$\{\hat{\partial}_{\text{\textcolor{brown}{supdn}[u][Latin][ua]}} \nabla_{\text{\textcolor{brown}{supup}[u][Latin][ub]}} T^d_{\text{\textcolor{brown}{supdn}[u][Latin][la]} \text{\textcolor{brown}{supup}[u][Latin][lb]} c \}$$

```

```
In[*]:= CD[ua, CD[ub, T[la, lb, lc, ud]]]
SP`markPairs[List@%, {}, {}] /. fixPrivateRule
```

```
Out[*]=

$$\nabla^a \nabla^b T^d_{abc}$$

```

```
Out[*]=

$$\{\nabla_{\text{\textcolor{brown}{supup}[u][Latin][ua]}} \nabla_{\text{\textcolor{brown}{supup}[u][Latin][ub]}} T^d_{\text{\textcolor{brown}{supup}[u][Latin][la]} \text{\textcolor{brown}{supup}[u][Latin][lb]} c \}$$

```

LD[v,T[indices]] where $\mathcal{L}_v g_{ab} \neq 0$:

```
In[*]:= CD[ua, LD[v, T[la, ud, le, lf]]] × A[ld]
SP`markPairs[List@@%, {}, {}] /. fixPrivateRule
```

```
Out[*]=

$$A_a \nabla^b \mathcal{L}_v T^a_{b\ ef}$$

```

```
Out[*]=

$$\{A_{\text{\textcolor{brown}{sdnup}[u][Latin][la]}}, \nabla_{\text{\textcolor{brown}{supdn}[u][Latin][ub]}} \mathcal{L}_v T^d_{\text{\textcolor{brown}{supdn}[u][Latin][lb]} \text{\textcolor{brown}{sdnup}[u][Latin][ua]} ef \}$$

```

연산자의 인자에 대한 문법 확인을 끌 수 있다:

```
In[*]:= CD[ua, LD[V, T[la, ud, le, lf]]] × A[ld]
SP`markPairs[List@@%, {}, {}]
```

Out[*]=

$$A_a \nabla^b \mathcal{L}_V T_b^a{}_{ef}$$

Msg: non -vector: V

Out[*]=

$$\{A_a, \nabla^b \mathcal{L}_V T_b^a{}_{ef}\}$$

```
In[*]:= SP`markPairs[List@@%, {}, {}, False] /. fixPrivateRule
```

Out[*]=

$$\{A^{\$dnup[u][Latin][la]}, \nabla^{\$updn[u][Latin][ub]} \mathcal{L}_V T^{\$updn[u][Latin][lb]}_{\$dnup[u][Latin][ua]}{}_{ef}\}$$

$\partial_a g_{bc} \neq 0$ 을 고려한 경우:

```
In[*]:= A[ud] × BD[ld, LD[v, T[la, lb, le, lf]]]
SP`markPairs[List@@%, {}, {}] /. fixPrivateRule
```

Out[*]=

$$A^c \partial_c \mathcal{L}_V T_{abef}$$

Out[*]=

$$\{A^{\$upup[u][Latin][uc]}, \hat{\partial}^{\$upup[u][Latin][lc]} \mathcal{L}_V T_{abef}\}$$

```
In[*]:= A[ud] × CD[lc, BD[ld, LD[v, T[la, lb, le, lf]]]]
SP`markPairs[List@@%, {}, {}] /. fixPrivateRule
```

Out[*]=

$$A^d \nabla_c \partial_d \mathcal{L}_V T_{abef}$$

Out[*]=

$$\{A^{\$upup[u][Latin][ud]}, \nabla_c \hat{\partial}^{\$upup[u][Latin][ld]} \mathcal{L}_V T_{abef}\}$$

```
In[*]:= A[ud] × CD[lc, BD[la, LD[v, T[ld, lb, le, lf]]]]
SP`markPairs[List@@%, {}, {}] /. fixPrivateRule
```

Out[*]=

$$A^d \nabla_c \partial_a \mathcal{L}_V T_{dbef}$$

Out[*]=

$$\{A^{\$updn[u][Latin][ud]}, \nabla_c \partial_a \mathcal{L}_V T^{\$updn[u][Latin][ld]}{}_{bef}\}$$

잘못된 입력인 경우에 잘못된 결과를 줄 수 있다:

```
In[*]:= BD[ua, CD[ub, T[lc, lb, ld, ue]]] × v[lb]
SP`markPairs[List@@%, {}, {}] /. fixPrivateRule
```

```
Out[*]=
```

$$\partial^a \nabla^b T_{cbd}^e v_b$$

```
Out[*]=
```

$$\left\{ \partial^a \nabla^{\text{supup}[u][\text{Latin}][ub]} T_c^{\text{supup}[u][\text{Latin}][lb]d} , v^{\text{supup}[u][\text{Latin}][lb]} \right\}$$

공변 도함수가 아니어도 dn-up 순서를 바꾸려면 option으로 지정한다.

```
In[*]:= CD[ua, BD[la, T[ub, ld, le, lf]]] × v[lb]
SP`markPairs[List@@%, {}, {}] /. fixPrivateRule
```

```
Out[*]=
```

$$\nabla^a \partial_a T_{def}^b v_b$$

```
Out[*]=
```

$$\left\{ \nabla^{\text{supup}[u][\text{Latin}][ua]} \hat{\partial}^{\text{supup}[u][\text{Latin}][la]} T_{\text{supdn}[u][\text{Latin}][ub]def}^b , v^{\text{supdn}[u][\text{Latin}][lb]} \right\}$$

```
In[*]:= SP`markPairs[List@@%%, {}, {CovDs → {CD, BD}}] /. fixPrivateRule
```

```
Out[*]=
```

$$\left\{ \nabla^{\text{supup}[u][\text{Latin}][ua]} \hat{\partial}^{\text{supup}[u][\text{Latin}][la]} T_{\text{supup}[u][\text{Latin}][ub]def}^b , v^{\text{supup}[u][\text{Latin}][lb]} \right\}$$

Kind에 대한 고려:

```
In[*]:= DefDerivativeOperator[CovD, "D", Latin];
Tdefine[T[lA], "*"]; Tdefine[v[lA]]
```

```
In[*]:= T[uC, lD, lE, lC]
SP`markPairs[List@%, {}, {}] /. fixPrivateRule (* No metric in Capital *)
```

```
Out[*]=
```

$$T_{DEA}^A$$

```
Out[*]=
```

$$\left\{ T_{\text{supdn}[u][\text{Capital}][uA]DE\text{supdn}[u][\text{Capital}][lA]}^A \right\}$$

```
In[*]:= T[uC, lD, lE, lA] × v[uA]
SP`markPairs[List@@%, {}, {}] /. fixPrivateRule (* No metric in Capital *)
```

```
Out[*]=
```

$$T_{DEA}^C v^A$$

```
Out[*]=
```

$$\left\{ T_{DE\text{snup}[u][\text{Capital}][lA]}^C , v^{\text{snup}[u][\text{Capital}][uA]} \right\}$$

Capital Kind를 DefaultKind로 설정한다:

```
In[*]:= SetDefaultKind[Capital]
```

```
In[*]:= T[uC, lD, lE, lC]
SP`markPairs[List@%, {}, {}] /. fixPrivateRule (* There is the 'Metricg' *)
```

```
Out[*]=
```

$$T_{DEA}^A$$

```
Out[*]=
```

$$\{T_{\$supup[u][Capital][uA]DE\$supup[u][Capital][lA]}^A\}$$

```
In[*]:= T[uC, lD, lE, lA] × v[uA]
SP`markPairs[List@@%, {}, {}] /.
  fixPrivateRule (* There is the 'Metricg' *)
```

```
Out[*]=
```

$$T_{DEA}^C v^A$$

```
Out[*]=
```

$$\{T_{DE\$dndn[u][Capital][lA]}^C, v_{\$dndn[u][Capital][uA]}^A\}$$

Capital Kind인 CD의 Kind가 Latin으로 입력된 경우:

```
In[*]:= BD[ua, CD[lb, T[uC, lD, lE, lC]]]
SP`markPairs[List@%, {}, {}]
```

```
Out[*]=
```

$$\partial^a \nabla_b T_{DEA}^A$$

General: lb is not a valid index.

```
Out[*]=
```

$$\{\partial^a \nabla_b T_{DEA}^A\}$$

```
In[*]:= SP`markPairs[List@%%, {}, {}, False] /. fixPrivateRule
```

```
Out[*]=
```

$$\{\partial^a \nabla_b T_{\$supup[u][Capital][uA]DE\$supup[u][Capital][lA]}^A\}$$

CD의 Kind가 Capital로 제대로 입력된 경우:

```
In[*]:= CD[lB, T[uA, lD, lE, lA]]
SP`markPairs[List@%, {}, {}]
```

```
Out[*]=
```

$$\nabla_B T_{DEA}^A$$

```
Out[*]=
```

$$\{\nabla_B T_{\$supup[m35][Capital][uA]DE\$supup[m35][Capital][lA]}^A\}$$

CovD의 Kind가 Latin으로 제대로 입력된 경우:

```
In[*]:= CovD[lb, T[uA, lD, lE, lA]]
SP`markPairs[List@%, {}, {}] /. fixPrivateRule
```

```
Out[*]=
```

$$D_b T_{DEA}^A$$

```
Out[*]=
```

$$\{D_b T_{\text{supup}[u][\text{Capital}][uA]DE\text{supup}[u][\text{Capital}][lA]}^A\}$$

CD의 Kind가 Capital로 제대로 입력된 경우:

```
In[*]:= CD[lB, T[uA, lD, lE, lC]] × v[lA]
SP`markPairs[List@@%, {}, {}] /. fixPrivateRule
```

```
Out[*]=
```

$$\nabla_B T_{DEC}^A \vee_A$$

```
Out[*]=
```

$$\{\nabla_B T_{\text{supup}[u][\text{Capital}][uA]DEC}^A, \vee_{\text{supup}[u][\text{Capital}][lA]}^A\}$$

CovD의 Kind가 Capital로 잘못 입력된 경우:

```
In[*]:= CovD[lB, T[uA, lD, lE, lC]] × v[lA]
SP`markPairs[List@@%, {}, {}]
```

```
Out[*]=
```

$$D_B T_{DEC}^A \vee_A$$

General: IB is not a valid index.

```
Out[*]=
```

$$\{D_B T_{DEC}^A, \vee_A\}$$

```
In[*]:= SP`markPairs[List@@%%, {}, {}, False] /. fixPrivateRule
```

```
Out[*]=
```

$$\{D_B T_{\text{supdn}[u][\text{Capital}][uA]DEC}^A, \vee_{\text{supdn}[u][\text{Capital}][lA]}^A\}$$

CovD의 Kind가 Latin으로 제대로 입력된 경우:

```
In[*]:= CovD[lb, T[uA, lD, lE, lC]] × v[lA]
SP`markPairs[List@@%, {}, {}] /. fixPrivateRule
```

```
Out[*]=
```

$$D_b T_{DEC}^A \vee_A$$

```
Out[*]=
```

$$\{D_b T_{\text{supdn}[u][\text{Capital}][uA]DEC}^A, \vee_{\text{supdn}[u][\text{Capital}][lA]}^A\}$$

CD의 Kind가 Greek로 잘못 입력된 경우:

```
In[*]:= BD[uA, CD[uμ, T[lA, lμ, lE, lF]]] × v[uC] × v[lC]
SP`markPairs[List@@%, {}, {}]
```

```
Out[*]=
```

$$\hat{\partial}^A \nabla^\alpha T_{A\alpha EF} v_B v^B$$

... General: $u\alpha$ is not a valid index.

```
Out[*]=
```

$$\{\hat{\partial}^A \nabla^\alpha T_{A\alpha EF}, v_B, v^B\}$$

CD의 Kind가 Capital로 제대로 입력된 경우:

```
In[*]:= BD[uA, CD[uG, T[lA, lB, lE, lF]]] × BD[uC, v[lD]] × v[uD]
SP`markPairs[List@@%, {}, {}] /. fixPrivateRule
```

```
Out[*]=
```

$$\hat{\partial}^C v_A \hat{\partial}^D \nabla^G T_{DBEF} v^A$$

```
Out[*]=
```

$$\{\hat{\partial}^C v_{\$dnup[u][Capital][lA]}, \hat{\partial}^{\$updn[u][Capital][uD]} \nabla^G T_{\$updn[u][Capital][lD]BEF}, v_{\$dnup[u][Capital][uA]}\}$$

CD의 Kind가 Latin으로 잘못 입력된 경우:

```
In[*]:= CD[uA, CD[ub, T[lA, lB, lE, lF]]] × BD[uC, v[lD]] × v[lC]
SP`markPairs[List@@%, {}, {}]
```

```
Out[*]=
```

$$\hat{\partial}^A v_D \nabla^B \nabla^a T_{BaEF} v_A$$

... General: ua is not a valid index.

```
Out[*]=
```

$$\{\hat{\partial}^A v_D, \nabla^A \nabla^a T_{AaEF}, v_A\}$$

```
In[*]:= SP`markPairs[List@@%%, {}, {}, False] /. fixPrivateRule
```

```
Out[*]=
```

$$\{\hat{\partial}^{\$upup[u][Capital][uA]} v_D, \nabla^{\$upup[u][Capital][uB]} \nabla^a T_{\$upup[u][Capital][lB]aEF}, v_{\$upup[u][Capital][lA]}\}$$

CD 인덱스인 ua 의 Kind가 Latin으로 잘못 입력되었음에도 ‘withCheck = False’로 syntax-checking을 꺾기 때문에 잘못된 결과값임을 알기 어렵다. 여기서, ∇^a 는 Latin Kind의 인덱스를 잘못 사용하였고, 따라서 Capital Kind의 공변 연산자로 취급할 수 없음에도 더미 인덱스 쌍인 $\{uB, lB\}$ 를 계량 텐서로 올리고 내릴 수 있는 것처럼 출력되었다. 따라서 syntax-checking을 하지 않으면 위험할 수 있다.

치명적 오류가 발생하는지 확인하자:

```
In[*]:= Tdefine[A[lA, lB], "-ba"]; Tdefine[S[lA, lB], "ba"]
```

```
In[*]:= S[lA, uB] × BD[lC, A[uA, lB]]
SP`markPairs[List@@%, {}, {}] /. fixPrivateRule
```

```
Out[*]=
```

$$\hat{\partial}_C A^A_B S^B_A$$

```
Out[*]=
```

$$\{\hat{\partial}_C A^{\text{supdn}[u][\text{Capital}][uA]\text{dnup}[u][\text{Capital}][lB]}, S^{\text{supdn}[u][\text{Capital}][lA]\text{dnup}[u][\text{Capital}][uB]}\}$$

```
In[*]:= S[lA, uB] × BD[lc, A[uA, lB]]
SP`markPairs[List@@%, {}, {}] /. fixPrivateRule
```

```
Out[*]=
```

$$\partial_C A^A_B S^B_A$$

```
Out[*]=
```

$$\{\partial_C A^{\text{supdn}[u][\text{Capital}][uA]\text{dnup}[u][\text{Capital}][lB]}, S^{\text{supdn}[u][\text{Capital}][lA]\text{dnup}[u][\text{Capital}][uB]}\}$$

```
In[*]:= S[lA, uB] × CD[lc, A[uA, lB]]
SP`markPairs[List@@%, {}, {}]
```

```
Out[*]=
```

$$\nabla_C A^A_B S^B_A$$

... **General:** lc is not a valid index.

```
Out[*]=
```

$$\{\nabla_C A^A_B, S^B_A\}$$

```
In[*]:= SP`markPairs[List@@%, {}, {}, False] /. fixPrivateRule
```

```
Out[*]=
```

$$\{\nabla_C A^{\text{supup}[u][\text{Capital}][uA]\text{dndn}[u][\text{Capital}][lB]}, S^{\text{supup}[u][\text{Capital}][lA]\text{dndn}[u][\text{Capital}][uB]}\}$$

CD 인덱스인 lc의 Kind가 Latin으로 잘못 입력되었음에도 syntax-checking을 꺾기 때문에 잘못된 결과값을 준다. 즉, ∇_C 는 Capital Kind의 인덱스에 대해 일반적으로 공변 연산자가 아님에도 더미 인덱스 쌍들 {uA, lA, lB, uB}를 계량 텐서로 올리고 내릴 수 있는 것처럼 출력되었다. 그렇게 되면 대칭에 의해 그 결과값은 0이 된다. 이렇게 명백히 잘못된 결과값을 얻으므로 syntax-checking을 끄면 대단히 위험하다. **checkOperatorArg** 함수로 연산자 인덱스의 적절함을 반드시 확인해야 한다.

CD 인덱스인 lC의 Kind가 Capital이어서 ∇_C 가 Capital Kind의 인덱스에 대해 공변 연산자가 되므로 문제가 없는 경우이다:

```
In[*]:= S[lA, uB] × CD[lC, A[uA, lB]]
SP`markPairs[List@@%, {}, {}] /. fixPrivateRule
```

```
Out[*]=
```

$$\nabla_C A^A_B S^B_A$$

```
Out[*]=
```

$$\{\nabla_C A^{\text{supup}[u][\text{Capital}][uA]\text{dndn}[u][\text{Capital}][lB]}, S^{\text{supup}[u][\text{Capital}][lA]\text{dndn}[u][\text{Capital}][uB]}\}$$

CovD 연산자는 Latin Kind이므로 입력에 문제가 없지만 D_C 가 Capital Kind의 인덱스에 대해 공변 연산자가

아니다:

```
In[*]:= S[lA, uB] × CovD[lA, A[uA, lB]]
SP`markPairs[List@@%, {}, {}] /. fixPrivateRule

Out[*]=

$$D_a A^A_B S_A^B$$


Out[*]=

$$\{D_a A^{\$supdn[u][Capital][uA]\$dnup[u][Capital][lB]}, S^{\$supdn[u][Capital][lA]\$dnup[u][Capital][uB]}\}$$

```

기본 설정으로 복귀한다:

```
In[*]:= SetDefaultKind[Latin] (* default *)

In[*]:= Tdefine[T, "*"]; Tdefine[v, 1]
```

DnUpPair

```
In[*]:= DnUpPair::usage

Out[*]=
DnUpPair[expr, opts] converts all possible up-dn index
pairs in 'expr' to dn-up pairs. This is equivalent to raising
the lower index and lowering the upper index of each pair.
```

```
In[*]:= Tdefine[Z, "*"]; Tdefine[v, 1]
```

인덱스 쌍의 순서를 (가능하면) Dn-Up으로 재조정한다. 옵션으로 HeadQs, CovDs가 있다.

인덱스 쌍이 한 텐서 안에 있는 경우

```
In[*]:= Z[ub, lc, ld, lb]
DnUpPair[%]

Out[*]=

$$Z^a_{cda}$$


Out[*]=

$$Z_{acd}^a$$

```

연산자가 있는 경우

```
In[*]:= BD[ua, CD[ub, Z[lA, lb, lc, ud]]]
DnUpPair[%]

Out[*]=

$$\partial^a \nabla^b Z_{abc}^d$$


Out[*]=

$$\partial^a \nabla_b Z_a^b c^d$$

```

```
In[*]:= CD[ua, CD[ub, Z[la, lb, lc, ud]]]
DnUpPair[%]
```

```
Out[*]=
```

$$\nabla^a \nabla^b Z_{abc}^d$$

```
Out[*]=
```

$$\nabla_a \nabla_b Z^{ab}_c{}^d$$

$\mathcal{L}_v g_{ab} \neq 0$ 인 경우:

```
In[*]:= CD[ua, LD[v, Z[la, ud, le, lf]]] × v[ld]
DnUpPair[%]
```

```
Out[*]=
```

$$\nabla^a \mathcal{L}_v Z_a{}^b{}_{ef} v_b$$

```
Out[*]=
```

$$\nabla^a \mathcal{L}_v Z_a{}^b{}_{ef} v_b$$

잘못된 입력:

```
In[*]:= BD[ua, CD[ub, Z[lc, lb, ld, ue]]] × v[lb]
DnUpPair[%]
```

```
Out[*]=
```

$$\partial^a \nabla^b Z_{cbd}{}^e v_b$$

... **Msg:** duplicated indices: {ua, ub, lc, lb, ld, ue, lb }

```
Out[*]=
```

$$\partial^a \nabla^b Z_{cbd}{}^e v_b$$

공변 도함수가 아니어도 dn-up 순서를 바꾸려면 option으로 지정하면 된다.

```
In[*]:= CD[ua, BD[la, Z[ub, ld, le, lf]]] × v[lb]
DnUpPair[%]
```

```
Out[*]=
```

$$\nabla^a \partial_a Z^b{}_{def} v_b$$

```
Out[*]=
```

$$\nabla_a \partial^a Z^b{}_{def} v_b$$

```
In[*]:= DnUpPair[%%, CovDs → {CD, BD}]
```

```
Out[*]=
```

$$\nabla_a \partial^a Z_{bdef} v^b$$

Kind에 대한 고려:

```
In[*]:= Tdefine[Z[la], "*"]; Tdefine[v[la]]
```

In[*]:= **BD[LB, CD[lA, Z[uC, lD, lE, lC]]] × v[lA]**
DnUpPair[%] (* No metric in Capital *)

Out[*]=

$$\hat{\partial}_B \nabla_a Z_{DEC}^C V_A$$

Out[*]=

$$\hat{\partial}_B \nabla_a Z_{DEC}^C V_A$$

Capital Kind를 DefaultKind로 설정한다:

In[*]:= **SetDefaultKind[Capital]**

In[*]:= **BD[LB, CD[lF, Z[uC, lD, lE, lC]]] × v[lA]**
DnUpPair[%]

Out[*]=

$$\hat{\partial}_B \nabla_F Z_{DEC}^C V_A$$

Out[*]=

$$\hat{\partial}_B \nabla_F Z_{CDE}^C V_A$$

Capital Kind인 CD 연산자에 Greek Kind의 인덱스를 사용한 경우:

In[*]:= **BD[uA, CD[uμ, Z[lA, lμ, lE, lF]]] × v[uC] × v[lC]**
DnUpPair[%]

Out[*]=

$$\hat{\partial}^A \nabla^\alpha Z_{A\alpha EF} V_B V^B$$

... General: uα is not a valid index.

Out[*]=

$$\hat{\partial}^A \nabla^\alpha Z_{A\alpha EF} V_B V^B$$

$\partial_A g_{BC} \neq 0$ 인 경우:

In[*]:= **BD[uA, CD[uB, Z[lA, lμ, lE, lF]]] × v[uC] × v[lC]**
DnUpPair[%]

Out[*]=

$$\hat{\partial}^A \nabla^B Z_{A\mu EF} V_C V^C$$

Out[*]=

$$\hat{\partial}^A \nabla^B Z_{A\mu EF} V_C V^C$$

$\nabla_A g_{BC} = 0$ 인 경우:

In[*]:= **CD**[uA, **CD**[uB, **Z**[lA, lμ, lE, lF]]] × v[uC] × v[lC]
DnUpPair[%]

Out[*]=

$$\nabla^A \nabla^B Z_{A\mu EF} v_C v^C$$

Out[*]=

$$\nabla_A \nabla^B Z_{\mu EF}^A v_C v^C$$

In[*]:= **CD**[uA, **CD**[uB, **Z**[lA, lB, lE, lF]]] × **BD**[uC, v[lD]] × v[uD]
DnUpPair[%]

Out[*]=

$$\hat{\partial}^C v_A \nabla^B \nabla^D Z_{BDEF} v^A$$

Out[*]=

$$\hat{\partial}^C v_A \nabla_B \nabla_D Z_{EF}^{BD} v^A$$

치명적 오류가 발생하는지 확인하자:

In[*]:= **Tdefine**[A[lA, lB], "-ba"]; **Tdefine**[S[lA, lB], "ba"]

In[*]:= **S**[lA, uB] × **BD**[lC, A[uA, lB]]
DnUpPair[%]

Out[*]=

$$\hat{\partial}_C A^A_B S_A^B$$

Out[*]=

$$\hat{\partial}_C A^A_B S_A^B$$

In[*]:= **S**[lA, uB] × **BD**[lC, A[uA, lB]]
DnUpPair[%]

Out[*]=

$$\partial_C A^A_B S_A^B$$

Out[*]=

$$\partial_C A^A_B S_A^B$$

In[*]:= **S**[lA, uB] × **CD**[lC, A[uA, lB]]
DnUpPair[%]

Out[*]=

$$\nabla_C A^A_B S_A^B$$

... General: lC is not a valid index.

Out[*]=

$$\nabla_C A^A_B S_A^B$$

In[*]:= **S**[**lA**, **uB**] **×** **CD**[**lC**, **A**[**uA**, **lB**]]
DnUpPair[%]

Out[*]=
 $\nabla_C A^A_B S^B_A$

Out[*]=
 $\nabla_C A_{AB} S^{AB}$

CovDs option으로 지정하는 연산자는 미리 정의되어 있어야 한다.

In[*]:= **DnUpPair**[**expr**, **CovDs** → {**nonOP**}]

Msg: not defined operator (s) {nonOP }

Out[*]=
expr

기본 설정으로 복귀한다:

In[*]:= **SetDefaultKind**[**Latin**]

UpDnPair

In[*]:= **UpDnPair::usage**

Out[*]=
UpDnPair[**expr**, **opts**] converts all possible dn-up index pairs in 'expr' to up-dn pairs. This is equivalent to lowering the upper index and raising the lower index of each pair.

In[*]:= **Tdefine**[**Z**, "*"]; **Tdefine**[**v**, "a"]

인덱스 쌍의 순서를 (가능하면) Up-Dn으로 재조정한다. 옵션으로 HeadQs, CovDs가 있다.

인덱스 쌍이 한 텐서 안에 있는 경우

In[*]:= **Z**[**lb**, **lc**, **ld**, **ub**]
UpDnPair[%]

Out[*]=
 Z_{acd}^a

Out[*]=
 Z^a_{cda}

연산자가 있는 경우

In[*]:= **BD**[la, CD[lb, Z[ua, ub, lc, ud]]]
UpDnPair[%]

Out[*]=

$$\partial_a \nabla_b Z_c^{ab\ d}$$

Out[*]=

$$\partial_a \nabla^b Z_{bc}^a{}^d$$

In[*]:= **CD**[la, CD[lb, Z[ua, ub, lc, ud]]]
UpDnPair[%]

Out[*]=

$$\nabla_a \nabla_b Z_c^{ab\ d}$$

Out[*]=

$$\nabla^a \nabla^b Z_{abc}{}^d$$

In[*]:= **CD**[la, LD[v, Z[ua, ld, le, lf]]] × v[ud]
UpDnPair[%]

Out[*]=

$$\nabla_a \mathcal{L}_v Z_{bef}^a{}^b v^b$$

Out[*]=

$$\nabla_a \mathcal{L}_v Z_{bef}^a{}^b v^b$$

잘못된 입력:

In[*]:= **BD**[la, CD[lb, Z[lc, ub, ld, ue]]] × v[ub]
UpDnPair[%]

Out[*]=

$$\partial_a \nabla_b Z_c^{b\ e} v^b$$

Msg: duplicated indices: {la, lb, lc, ub, ld, ue, ub }

Out[*]=

$$\partial_a \nabla_b Z_c^{b\ e} v^b$$

공변 도함수가 아니어도 dn-up 순서를 바꾸려면 option으로 지정하면 된다.

In[*]:= **CD**[la, BD[ua, Z[lb, ld, le, lf]]] × v[ub]
UpDnPair[%]

Out[*]=

$$\nabla_a \partial^a Z_{bdef} v^b$$

Out[*]=

$$\nabla^a \partial_a Z_{bdef} v^b$$

In[*]:= UpDnPair[%%, CovDs → {CD, BD}]

Out[*]=

$$\nabla^a \partial_a Z_{\text{def}}^b V_b$$

TindexSort

In[*]:= TindexSort::usage

Out[*]=

TindexSort[expr] sorts the indices of a tensor expression into a canonical order based on its symmetries.

텐서 표현을 구성하는 각각의 텐서에 독립적으로 작용하여 인덱스를 정렬한다.

텐서 인덱스의 순열 대칭 때문에 자동으로 0이 되는가를 확인한다. 만일 0이 아니면 MakePermGroup 함수를 이용하여 **모든 가능한** 인덱스 표현을 얻고, IndexOrderedQ 함수를 이용하여 최소가 되는 인덱스 표현을 구한다. (참고: 각각 한 개의 텐서만을 대상으로 하기 때문에 과도한 대칭에 의한 연산 시간의 지수적 증가를 걱정할 필요는 없다.)

In[*]:= Tdefine[ε, "*-"]; Tdefine[A, "-ba"]; Tdefine[S, "ba"]

반대칭 텐서:

In[*]:= {A[la, lb], A[lb, la], A[ua, la]}

TindexSort /@ %

Out[*]=

$$\{A_{ab}, A_{ba}, A^a_a\}$$

Out[*]=

$$\{A_{ab}, -A_{ab}, 0\}$$

In[*]:= {ε[], ε[la], ε[lb, la], ε[lc, lb, la], ε[ua, lb, la]}

TindexSort /@ %

Out[*]=

$$\{\epsilon, \epsilon_a, \epsilon_{ba}, \epsilon_{cba}, \epsilon^a_{ba}\}$$

Out[*]=

$$\{\epsilon, \epsilon_a, -\epsilon_{ab}, -\epsilon_{abc}, 0\}$$

반대칭 텐서의 성분 인덱스:

```
In[*]:= {A[-2, -1], A[1, -1], A[1, 1], A[-1, -1], A[2, 1]}
TindexSort /@ %
```

```
Out[*]=
```

$$\{A_{21}, A^1_{-1}, A^{11}, A_{11}, A^{21}\}$$

```
Out[*]=
```

$$\{-A_{12}, -A_1^1, 0, 0, -A^{12}\}$$

대칭 텐서:

```
In[*]:= {S[la, lb], S[lb, la], S[ua, la]}
TindexSort /@ %
```

```
Out[*]=
```

$$\{S_{ab}, S_{ba}, S^a_a\}$$

```
Out[*]=
```

$$\{S_{ab}, S_{ab}, S^a_a\}$$

```
In[*]:= S[lc, la, lb] (* S는 rank-2 텐서 *)
% // TindexSort
```

```
Out[*]=
```

$$S_{cab}$$

```
Out[*]=
```

$$S_{cab}$$

```
In[*]:= {S[lc, lc], S[uc, lc]}
% // TindexSort
```

```
Out[*]=
```

$$\{S_{cc}, S^a_a\}$$

```
Out[*]=
```

$$\{S_{cc}, S^a_a\}$$

BD 연산자의 인덱스 대칭:

```
In[*]:= {BD[lb, ua, A[ub, la]], BD[ub, la, A[lb, ua]], BD[lb, la, A[ub, ua]]}
TindexSort /@ %
```

```
Out[*]=
```

$$\{\partial_a \partial^b A^a_b, \partial^a \partial_b A_a^b, \partial_a \partial_b A^{ab}\}$$

```
Out[*]=
```

$$\{\partial_a \partial^b A^a_b, \partial^a \partial_b A_a^b, 0\}$$


```
In[*]:= {BD[lb, ua, S[ub, la]], BD[ub, la, S[lb, ua]], BD[lb, la, S[ub, ua]]}
TindexSort /@ %
```

```
Out[*]= {∂a∂bSab, ∂a∂bSab, ∂a∂bSab}
```

```
Out[*]= {∂a∂bSab, ∂a∂bSab, ∂a∂bSab}
```

CD 연산자의 인덱스 대칭:

```
In[*]:= Tdefine[f[]]
```

```
In[*]:= CD[lb, la, f[]]
% // TindexSort
```

```
Out[*]= ∇b∇af
```

```
Out[*]= ∇a∇bf
```

```
In[*]:= CD[lB, lA, f[]]
% // TindexSort
```

```
Out[*]= ∇B∇Af
```

... General: lA is not a valid index.

```
Out[*]= ∇B∇Af
```

```
In[*]:= BD[lB, lA, f[]]
% // TindexSort
```

```
Out[*]= ∂B∂Af
```

```
Out[*]= ∂B∂Af
```

Rules for Simplification

BDinvRule

`In[*]:= BDinvRule::usage`

`Out[*]=`

`BDinvRule[metric]` provides a transformation rule for the basis derivative of an inverse metric. The metric defaults to `Metricg`.

기본으로 `Metricg`에 대한 규칙이다:

`In[*]:= {BD[la, Metricg[ub, uc]], BD[la, Metricg[lb, lc]]}
% /. BDinvRule[]`

`Out[*]=`

$\{\partial_a g^{bc}, \partial_a g_{bc}\}$

`Out[*]=`

$\{-\partial_a g_{de} g^{bd} g^{ce}, \partial_a g_{bc}\}$

특정한 rank-2 대칭 텐서에 대한 규칙으로 동작하게 할 수 있다:

`In[*]:= {BD[la, Metricg[ub, uc]], BD[la, Metricg[lb, lc]]}
% /. BDinvRule[Metricg]`

`Out[*]=`

$\{\partial_a g^{bc}, \partial_a g_{bc}\}$

`Out[*]=`

$\{-\partial_a g_{de} g^{bd} g^{ce}, \partial_a g_{bc}\}$

KdeltaSumRule

`In[*]:= KdeltaSumRule::usage`

`Out[*]=`

`KdeltaSumRule[kind]` provides a rule to replace the trace of a Kronecker delta (e.g., `Kdelta[la,ua]`) with the dimension of the specified kind. The kind defaults to `DefaultKind`.

`DefaultKind`의 인덱스에 대해 동작한다:

```
In[*]:= Kdelta[la, lb]
% /. KdeltaSumRule[]
```

```
Out[*]=
```

$$\delta_{ab}$$

```
Out[*]=
```

$$\delta_{ab}$$

```
In[*]:= Kdelta[la, ua]
% /. KdeltaSumRule[]
```

```
Out[*]=
```

$$\delta_a^a$$

```
Out[*]=
```

```
GetDimension[Latin]
```

```
In[*]:= Kdelta[ua, la]
% /. KdeltaSumRule[]
```

```
Out[*]=
```

$$\delta_a^a$$

```
Out[*]=
```

```
GetDimension[Latin]
```

```
In[*]:= Kdelta[lA, uA]
% /. KdeltaSumRule[]
```

```
Out[*]=
```

$$\delta_A^A$$

```
Out[*]=
```

$$\delta_A^A$$

특정한 Kind의 인덱스에 대해 동작하도록 지정할 수 있다:

```
In[*]:= Kdelta[lA, uA]
% /. KdeltaSumRule[Capital]
```

```
Out[*]=
```

$$\delta_A^A$$

```
Out[*]=
```

```
GetDimension[Capital]
```

EpsilonProductRule

In[*]:= **EpsilonProductRule::usage**

Out[*]=

EpsilonProductRule[kind] provides a rule to simplify the product of two Levi-Civita tensors for the specified kind. The kind defaults to DefaultKind.

In[*]:= **Format[GetSig[Latin]] = s; Format[GetDimension[Latin]] = n;**

DefaultKind의 인덱스에 대해 동작한다:

In[*]:= **Epsilon[ua, ub] × EpsilonGreek[lμ, lv]**
% /. EpsilonProductRule[]

Out[*]=

$$\epsilon^{ab} \in [\text{Greek}]_{\mu\nu}$$

Out[*]=

$$\epsilon^{ab} \in [\text{Greek}]_{\mu\nu}$$

In[*]:= **Epsilon[ua, ub] × Epsilon[lc, ld]**
% /. EpsilonProductRule[]

Out[*]=

$$\epsilon_{cd} \epsilon^{ab}$$

Out[*]=

$$-(-1)^s \delta_c^b \delta_d^a + (-1)^s \delta_c^a \delta_d^b$$

In[*]:= **Epsilon[ua, ub, up] × Epsilon[lc, ld, lp]**
% /. EpsilonProductRule[]

Out[*]=

$$\epsilon_{cde} \epsilon^{abe}$$

Out[*]=

$$2(-1)^s \delta_c^b \delta_d^a - (-1)^s n \delta_c^b \delta_d^a - 2(-1)^s \delta_c^a \delta_d^b + (-1)^s n \delta_c^a \delta_d^b$$

In[*]:= **% // Simplify**

Out[*]=

$$-(-1)^s (-2+n) \delta_c^b \delta_d^a + (-1)^s (-2+n) \delta_c^a \delta_d^b$$

특정한 Kind의 인덱스에 대해 동작하도록 지정할 수 있다:

In[*]:= **Format[GetSig[Greek]] = ss; Format[GetDimension[Greek]] = nn;**

In[*]:= **DefMetric[g, Greek]**

```
In[*]:= EpsilonGreek[uμ, uv] × EpsilonGreek[lρ, lσ]
% /. EpsilonProductRule[Greek]
```

Out[*]=

$$\in [\text{Greek}]_{\rho\sigma} \in [\text{Greek}]^{\mu\nu}$$

Out[*]=

$$-(-1)^{ss} \delta_{\rho}^{\nu} \delta_{\sigma}^{\mu} + (-1)^{ss} \delta_{\rho}^{\mu} \delta_{\sigma}^{\nu}$$

```
In[*]:= Format[GetSig[Greek]] = .; Format[GetDimension[Greek]] = .;
```

사용 예제:

```
In[*]:= SetDimension[4]; SetSig[1];
Tdefine[A, "*-"]
```

```
In[*]:= DualStar[A[ua, ub], {lc, ld}]
DualStar[%, {ue, uf}]
```

Out[*]=

$$\frac{1}{2} A^{ab} \epsilon_{abcd}$$

Out[*]=

$$\frac{1}{4} A^{ab} \epsilon_{abcd} \epsilon^{cdef}$$

```
In[*]:= % /. EpsilonProductRule[]
% // TindexSort
```

Out[*]=

$$-\frac{1}{2} A^{ef} + \frac{A^{fe}}{2}$$

Out[*]=

$$-A^{ef}$$

```
In[*]:= DualStar[A[ua, ub, uc], {ld}]
DualStar[%, {ue, uf, ug}]
```

Out[*]=

$$\frac{1}{6} A^{abc} \epsilon_{abcd}$$

Out[*]=

$$\frac{1}{6} A^{abc} \epsilon_{abcd} \epsilon^{defg}$$

```
In[*]:= % /. EpsilonProductRule[]
% // TindexSort
```

```
Out[*]=
```

$$\frac{1}{6} A^{efg} - \frac{1}{6} A^{egf} - \frac{1}{6} A^{feg} + \frac{1}{6} A^{fge} + \frac{1}{6} A^{gef} - \frac{1}{6} A^{gfe}$$

```
Out[*]=
```

A^{efg}

기본 설정으로 복귀한다:

```
In[*]:= ClearDimension[];
ClearSig[];
```

Tsimplify

Implementation

```
In[*]:= Tsimplify::usage
```

```
Out[*]=
```

Tsimplify[expr, opts] simplifies a tensor expression by applying symmetries and contracting indices. It is primarily designed for symmetric metrics.

Tsimplify 함수는 대칭 계량 텐서를 갖는 시공간에서의 텐서 표현을 Mathematica의 TensorReduce를 이용하여 표준화한다. Option으로는 HeadQs와 CovDs가 있다.

Tsimplify는 CovDs 옵션에 있는 기호가 정의된 연산자인가를 확인한다. 그 후, \$AssumSymL = {}로 설정한 Block 안에서 tSimplifyAux[]를 호출한다.

tSimplifyAux

ExpandObject로 텐서 표현을 전개한 후 ForEachTerm을 이용하여 임의의 표현을 단일항의 조합으로 분해한 후, 각각의 항에 tReduceTerm을 함수를 적용한다.

tReduceTerm

SplitTerm을 이용하여 비텐서항과 텐서항을 추출한 후, 텐서항에 xSort를 적용한다. 그 후 tReduceTermProcess를 호출한다.

tReduceTermProcess

인덱스가 없는 항은 비텐서항으로 간주한다. 따라서 앞으로는 인덱스가 있는 항만을 다룬다고 가정한다.

첫 번째 인자는 텐서항에 xSort를 적용한 결과인 flatTerm이고, 두 번째와 세 번째 인자는 SplitTerm의 적용 결과인 ordTerm과 oTerm이다. 나머지는 cOptL과 verb이다.

flatTerm에 xSymmetryOf를 적용하여 xSym을 얻는다.

xSym으로부터 텐서항을 구성하는 텐서의 리스트인 oTermL을 얻는다.

텐서항의 축약된 인덱스들 중에서 **계량 텐서가 대칭이 아닌 인덱스**가 있으면 더 이상의 연산은 중지하고 입력값을 그대로 돌려 준다. (2026.01.24 현재 대칭인 계량 텐서만 다루므로 필요없는 코드)

oTermL에 markPairs를 적용하여 markedTermL을 얻는다.

reduceSymByMark를 호출하여 \$supdn 또는 \$dnup으로 마킹된 축약된 인덱스와 관련한 대칭을 xSym으로부터 제거한다.

oTermL의 각 텐서에 flattenObject를 적용하여 텐서곱의 이름을 병합한다. (이 때 \$flattenOther에 대한 예외 상황을 고려한다.)

prepareSymbolicTensor를 호출하여 \$AssumSymL에 name 항목을 등록한다.

doTensorReduce를 호출하여 {sign, result, sortedFreeL}을 돌려 받는다. 만일 result 값이 0이면 0을 돌려 주고 종료한다. 유사하게 sortedFreeL이 Null이면 내부 오류임을 의미하고 입력값을 그대로 돌려 주고 종료한다.

flatTerm에 metricStatesOf를 적용한 결과를 두 번째 인자로 하여 postTensorContract를 호출한다. 그 결과 indexL과 result를 얻는다.

두 번째 인자를 sortedFreeL, 세 번째 인자를 sign으로 하여 postTensorTranspose를 호출하여 새로운 sortedFreeL을 얻는다.

indexL에서 값이 0인 원소를 sortedFreeL의 해당하는 원소로 수정한다.

결과값으로 sign * (indexL의 순서로 정렬된 텐서곱)을 돌려 준다.

reduceSymByMark: 두 번째 인자인 markedTermL에서 \$supup이나 \$dndn으로 마킹되지 않은 축약된 인덱스를 찾고, 그 인덱스와 관련한 대칭을 첫 번째 인자인 gs로부터 제거한다.

prepareSymbolicTensor

첫 번째 인자인 name이 \$AssumSymL에 등록되지 않았으면, 두 번째 인자인 indexL에 대한 kindL과 세 번째 인자인 gs를 \$AssumSymL에 새로 등록한다.

doTensorReduce

name, xSym, indexL을 인자로 하여 Mathematica 내장 함수인 TensorReduce를 적용한다.

반복된 성분 인덱스가 있는 경우 indexL에서 그 인덱스들의 위치로 구성된 repeatL을 구한다. 만일 repeatedIndicesZeroQ를 호출하여 repeatL에 있는 위치의 대칭들에 반대칭이 포함되어 있다는 것을 알게 되면 결과값으로 0을 돌려 주고 종료한다.

repeatL로부터 유일한 기호로 구성된 uniqueL을 얻고, repeatL에 있는 위치에 해당되는 indexL의 인덱스들도 바꾼다. uniqueL의 구조는 {‘유일한 심볼’[실제 성분 인덱스 값],...}이다. 예를 들면, R[-4,-4]의 경우 uniqueL은 {zz123[-4], zz124[-4]}이다. 결국 이러한 과정을 거친 반복된 성분 인덱스들은 TensorReduce에서 free 인덱스들로 간주된다.

indexL에 TakePairs를 적용하여 축약된 인덱스 pairL을 구하고, 유사하게 sortIndicesProper를 적용하여 free 인덱스를 정렬한 sortedIndexL를 구한다.

sortedIndexL에 해당하는 인덱스를 갖는 표현을 얻기 위해 TensorTranspose를 적용한다.

pairL로부터 얻은 표현 contL을 이용하여 TensorContract를 적용한다.

prepareSymbolicTensor에서 얻은 \$AssumSymL을 이용하여 TensorReduce를 적용한다. 그 결과가 0이면 두 번째 원소가 0인 리스트를 돌려 주고 종료한다.

TensorReduce의 결과값 result에서 부호 sign을 제거하고, free 인덱스들을 정렬한 sortedIndexL에 uniqueL을 이용하여 원래의 반복된 성분 인덱스를 복구한다. 결과적으로 {sign, result, sortedIndexL}을 돌려 준다.

repeatedIndicesZeroQ: 세 번째 인자인 repeatL에 반대칭 인덱스가 포함되었는가를 판단한다.

postTensorContract

세 번째 인자인 indexL의 크기와 동일하고 모든 원소가 0인 idxL을 도입하고, TakePairs를 적용하여 pairL을 얻는다.

첫 번째 인자인 result (doTensorReduce 결과 리스트의 두 번째 원소)로부터 contL을 구한다.

contL의 각 원소마다 새로운 더미 인덱스 쌍 dumpairL을 생성하고, 두 번째 인자인 mStateL의 정보를 이용하여 moveQ == +1이면 dumpairL을 {dn,up}, 그렇지 않으면 {up,dn}으로 바꾸어 idxL을 업데이트한다. 결과적으로 idxL의 원소가 0이면 free 인덱스이고 아니면 mStateL에 따라 {dn,up} 또는 {up,dn} 순서를 갖는 축약된 인덱스이다.

돌려 주는 리스트의 첫 번째 원소는 idxL이고, 두 번째 원소는 result에서 TensorContract Head를 제거한 표현이다.

postTensorTranspose

첫 번째 인자 result는 postTensorContract 결과 리스트의 두 번째 원소이고, 두 번째 인자는 doTensorReduce 결과 리스트의 세 번째 원소인 sortedIndexL이다.

첫 번째 인자 result로부터 transL을 구한다.

transL의 Imag 표현을 구하고, InversePerm으로 역순열을 구한다. 이것을 PermuteList로 sortedIndexL에 적용한 결과를 돌려 준다.

Examples

```
In[*]:=
Tdefine[T, "*"];
Tdefine[f[]];
Tdefine[v, "a"];
Tdefine[A, "-ba"];
Tdefine[B, "ab"];
Tdefine[S, "ba"];
Tdefine[F, "-ba"]
```

Single Tensor

```
In[*]:=
{A[ua, ub], A[lb, ub], A[lb, la], S[ub, ua], S[ua, la]}
% // Tsimply
```

Out[*]=

```
{Aab, Aaa, Aba, Sba, Saa}
```

Out[*]=

```
{Aab, 0, -Aab, Sab, Saa}
```



```
In[*]:= {S[1, 1], F[1, 1], F[1, -1]}
% // Tsimplify
```

```
Out[*]= {S11, F11, F11}
```

```
Out[*]= {S11, 0, -F11}
```

```
In[*]:= {CD[la, lb, A[ua, ub]], CD[la, lb, S[ua, ub]], CD[la, lb, A[ua, ub]]}
% // Tsimplify
```

```
Out[*]= {∇a∇bAab, ∇a∇bSab, ∇a∇cAab}
```

```
Out[*]= {∇a∇bAab, ∇a∇bSab, -∇a∇cAba}
```

```
In[*]:= BD[la, lb, A[ua, lb]] (* errObject *)
% // Tsimplify
```

```
Out[*]= ∂a∂bAab
```

```
Out[*]= ∂a∂bAab
```

```
In[*]:= {BD[la, ub, A[ua, lb]], BD[la, lb, A[ub, ua]], BD[ua, lb, A[ub, la]]}
% // Tsimplify
```

```
Out[*]= {∂a∂bAab, ∂a∂bAba, ∂a∂bAba}
```

```
Out[*]= {∂a∂bAab, 0, ∂a∂bAba}
```

Two Tensors

$$A_{[ab]}S^{(ab)} = 0?$$

```
In[*]:= A[la, lb] × S[ua, ub]
% // Tsimplify
```

```
Out[*]= Aab Sab
```

```
Out[*]= 0
```

```
In[*]:= {S[la, lb] × BD[lc, A[ua, ub]],  
         S[la, ub] × CD[lc, A[ua, lb]], S[la, ub] × BD[lc, A[ua, lb]]}  
% // Tsimplify
```

```
Out[*]= {∂cAabSab, ∇cAabSab, ∂cAabSab}
```

```
Out[*]= {0, 0, ∂cAabSab}
```

```
In[*]:= expr = S[la, ub] × BD[lc, A[ua, lb]]  
Tsimplify[expr]  
Tsimplify[expr, CovDs → {CD, BD}]
```

```
Out[*]= ∂cAabSab
```

```
Out[*]= ∂cAabSab
```

```
Out[*]= 0
```

$$A^{[ab]}\nabla_a\nabla_b f = 0?$$

```
In[*]:= TorsionFreeQ[CD] = False;
```

```
In[*]:= expr = {CD[la, lb, f[]] × A[ua, ub], CD[la, lb, S[lc, ld]] × A[ua, ub]}  
expr // Tsimplify
```

```
Out[*]= {Aab∇a∇bf, Aab∇a∇bScd}
```

```
Out[*]= {Aab∇a∇bf, Aab∇a∇bScd}
```

```
In[*]:= TorsionFreeQ[CD] = True;  
expr // Tsimplify
```

```
Out[*]= {0, Aab∇a∇bScd}
```

```
In[*]:= expr = {CD[la, ub, f[]] × A[ua, lb], BD[lc, CD[la, ub, f[]]] × A[ua, lb]}
Timplify[expr]
Timplify[expr, CovDs → {CD, BD}]
```

Out[*]=

$$\{A^a_b \nabla_a \nabla^b f, A^a_b \partial_c \nabla_a \nabla^b f\}$$

Out[*]=

$$\{0, A^a_b \partial_c \nabla_a \nabla^b f\}$$

Out[*]=

$$\{0, 0\}$$

$A^{[ab]} \partial_a \partial_b T_{cd} = 0?$

```
In[*]:= expr = {BD[la, lb, f[]] × A[ua, ub], BD[la, lb, S[uc, ud]] × A[ua, ub],
  BD[lc, ua, lb, S[ud, ue]] × A[la, ub], CD[lc, BD[ua, lb, S[ud, ue]]] × A[la, ub],
  CD[lc, BD[la, ub, S[ud, ue]]] × A[ua, lb]}
expr // Timplify (* The fourth should be zero! *)
```

Out[*]=

$$\{A^{ab} \partial_a \partial_b f, A^{ab} \partial_a \partial_b S^{cd}, A_a^b \partial_c \partial^a \partial_b S^{de}, A_a^b \nabla_c \partial^a \partial_b S^{de}, A_a^b \nabla_c \partial_a \partial^b S^{de}\}$$

Out[*]=

$$\{0, 0, A_a^b \partial_c \partial^a \partial_b S^{de}, A_a^b \nabla_c \partial^a \partial_b S^{de}, A_a^b \nabla_c \partial_a \partial^b S^{de}\}$$

```
In[*]:= PutMetric[%[[4]], ua] // Absorbg
```

Out[*]=

$$A^{ab} \nabla_c \partial_a \partial_b S^{de}$$

```
In[*]:= % // Timplify
```

Out[*]=

$$0$$

Kind

```
In[*]:= SetDefaultKind[Capital]
```

```
In[*]:= Tdefine[A[la, lb], "-ba"]; Tdefine[S[la, lb], "ba"]
```

```
In[*]:= {S[la, ub] × CD[lc, A[ua, lb]], S[la, uB] × CD[lc, A[ua, lb]],
  S[la, uB] × CD[lc, A[ua, lb]], S[la, uB] × CD[lc, A[ua, lb]]}
% // Timplify
```

Out[*]=

$$\{\nabla_c A^a_b S_a^b, \nabla_c A^a_A S_a^A, \nabla_c A^a_A S_a^A, \nabla_c A^A_B S_A^B\}$$

Out[*]=

$$\{\nabla_c A^a_b S_a^b, \nabla_c A^a_A S_a^A, \nabla_c A^a_A S_a^A, 0\}$$

```
In[*]:= {S[lA, uB] × CD[lc, A[uA, lB]],
          S[lA, uB] × CD[lC, A[uA, lB]], S[lA, uB] × BD[lc, A[uA, lB]]}
% // Tsimplify
```

```
Out[*]=
```

$$\{\nabla_c A_B^A S_A^B, \nabla_C A_B^A S_A^B, \partial_c A_B^A S_A^B\}$$

```
Out[*]=
```

$$\{\nabla_c A_B^A S_A^B, 0, \partial_c A_B^A S_A^B\}$$

```
In[*]:= {CD[la, lb, f[]] × A[ua, ub], CD[lA, lB, f[]] × A[uA, uB]}
% // Tsimplify
```

```
Out[*]=
```

$$\{A^{ab} \nabla_a \nabla_b f, A^{AB} \nabla_A \nabla_B f\}$$

```
Out[*]=
```

$$\{A^{ab} \nabla_a \nabla_b f, 0\}$$

```
In[*]:= {CD[la, ub, f[]] × A[ua, lb], CD[la, uB, f[]] × A[ua, lB]}
% // Tsimplify
```

```
Out[*]=
```

$$\{A^a_b \nabla_a \nabla^b f, A^a_A \nabla_a \nabla^A f\}$$

```
Out[*]=
```

$$\{A^a_b \nabla_a \nabla^b f, A^a_A \nabla_a \nabla^A f\}$$

```
In[*]:= {CD[lA, uB, f[]] × A[uA, lB], BD[lc, CD[lA, uB, f[]]] × A[uA, lB]}
% // Tsimplify
```

```
Out[*]=
```

$$\{A^A_B \nabla_A \nabla^B f, A^A_B \partial_c \nabla_A \nabla^B f\}$$

```
Out[*]=
```

$$\{0, A^A_B \partial_c \nabla_A \nabla^B f\}$$

```
In[*]:= Off[CoordinateBasisFlag[Capital]]
```

```
In[*]:= {BD[la, lb, S[lc, ld]] × A[ua, ub],
          BD[lA, lB, S[lc, ld]] × A[uA, uB], BD[lA, lB, S[lC, lD]] × A[uA, uB]}
% // Tsimplify
```

```
Out[*]=
```

$$\{A^{ab} \partial_a \partial_b S_{cd}, A^{AB} \hat{\partial}_A \hat{\partial}_B S_{cd}, A^{AB} \hat{\partial}_A \hat{\partial}_B S_{CD}\}$$

```
Out[*]=
```

$$\{A^{ab} \partial_a \partial_b S_{cd}, A^{AB} \hat{\partial}_A \hat{\partial}_B S_{cd}, A^{AB} \hat{\partial}_A \hat{\partial}_B S_{CD}\}$$

```
In[*]:= On[CoordinateBasisFlag[Capital]]
```

In[*]:=	BD[lA, lB, S[lC, lD]] × A[uA, uB] % // Tsimplify
Out[*]=	$A^{AB} \partial_A \partial_B S_{CD}$
Out[*]=	$\emptyset$
In[*]:=	{BD[ua, lb, S[lc, ld]] × A[la, ub], BD[ua, lB, S[lc, ld]] × A[la, uB]} % // Tsimplify
Out[*]=	$\{A_a^b \partial^a \partial_b S_{cd}, A_a^A \partial^a \partial_A S_{cd}\}$
Out[*]=	$\{A_a^b \partial^a \partial_b S_{cd}, A_a^A \partial^a \partial_A S_{cd}\}$
In[*]:=	{BD[uA, lB, S[lc, ld]] × A[lA, uB], BD[uA, lB, S[lC, lD]] × A[lA, uB], BD[lA, uB, S[lc, ld]] × A[uA, lB]} % // Tsimplify
Out[*]=	$\{A_A^B \partial^A \partial_B S_{cd}, A_A^B \partial^A \partial_B S_{CD}, A_A^A \partial_A \partial^B S_{cd}\}$
Out[*]=	$\{A_A^B \partial^A \partial_B S_{cd}, A_A^B \partial^A \partial_B S_{CD}, A_A^A \partial_A \partial^B S_{cd}\}$
In[*]:=	{BD[la, BD[uA, lB, S[lc, ld]]] × A[lA, uB], CD[la, BD[uA, lB, S[lc, ld]]] × A[lA, uB], CD[la, BD[uA, lB, S[lC, lD]]] × A[lA, uB], CD[lE, BD[uA, lB, S[lC, lD]]] × A[lA, uB]} % // Tsimplify
Out[*]=	$\{A_A^B \partial_a \partial^A \partial_B S_{cd}, A_A^B \nabla_a \partial^A \partial_B S_{cd}, A_A^B \nabla_a \partial^A \partial_B S_{CD}, A_A^B \nabla_E \partial^A \partial_B S_{CD}\}$
Out[*]=	$\{A_A^B \partial_a \partial^A \partial_B S_{cd}, A_A^B \nabla_a \partial^A \partial_B S_{cd}, A_A^B \nabla_a \partial^A \partial_B S_{CD}, A_A^B \nabla_E \partial^A \partial_B S_{CD}\}$
In[*]:=	SetDefaultKind[Latin] (* default *) Tdefine[A, "-ba"]; Tdefine[S, "ba"]

Three and More Tensors

$$A^{[ab]} v_a v_b = 0$$

```
In[*]:= {A[ua, ub] × v[la] × v[lb], A[ua, lb] × v[la] × v[ub],  
A[ua, lb] × v[la] × CD[lc, v[ub]], A[ua, lb] × CD[uc, v[la]] × CD[lc, v[ub]]}  
% // Tsimplify
```

Out[*]=

$$\{A^{ab} v_a v_b, A^a_b v_a v^b, A^a_b \nabla_c v^b v_a, A^a_b \nabla_c v^b \nabla^c v_a\}$$

Out[*]=

$$\{0, 0, A_{ab} \nabla_c v^b v^a, 0\}$$

```
In[*]:= {A[la, lb] × B[ua, lc] × B[ub, uc], A[la, lb] × B[ua, lc] × B[ub, ud]}  
% // Tsimplify
```

Out[*]=

$$\{A_{ab} B^a_c B^{bc}, A_{ab} B^a_c B^{bd}\}$$

Out[*]=

$$\{0, A_{ab} B^a_c B^{bd}\}$$

```
In[*]:= {A[ua, ub] × BD[ld, B[la, lc]] × BD[ud, B[lb, uc]],  
A[ua, ub] × CD[ld, B[la, lc]] × CD[ud, B[lb, uc]],  
A[ua, ub] × CD[ld, B[la, lc]] × B[lb, uc]}  
% // Tsimplify
```

Out[*]=

$$\{A^{ab} \partial_c B_{ad} \partial^c B_b^d, A^{ab} \nabla_c B_{ad} \nabla^c B_b^d, A^{ab} B_b^c \nabla_d B_{ac}\}$$

Out[*]=

$$\{A^{ab} \partial_c B_{ad} \partial^c B_b^d, 0, -A_{ab} B^a_c \nabla_d B^{bc}\}$$

$$A^{[ab]} \nabla_a v_c \nabla_b v^c = 0$$

```
In[*]:= {A[ua, lb] × BD[la, f[]] × BD[ub, f[]], A[ua, ub] × BD[la, v[lc]] × BD[lb, v[uc]],  
A[ua, ub] × CD[la, v[lc]] × CD[lb, v[uc]]}  
% // Tsimplify
```

Out[*]=

$$\{A^a_b \partial_a f \partial^b f, A^{ab} \partial_a v_c \partial_b v^c, A^{ab} \nabla_a v_c \nabla_b v^c\}$$

Out[*]=

$$\{0, A^{ab} \partial_a v_c \partial_b v^c, 0\}$$

```
In[*]:= A[ua, ub] × BD[la, v[ud]] × BD[lb, v[ue]] × Metricg[ld, le]  
% // Tsimplify
```

Out[*]=

$$A^{ab} \partial_a v^c \partial_b v^d g_{cd}$$

Out[*]=

$$0$$

```
In[*]:= {A[ua, ub] × BD[la, B[lc, ld]] × BD[lb, B[uc, ud]],
        A[ua, ub] × CD[la, B[lc, ld]] × CD[lb, B[uc, ud]],
        A[ua, ub] × CD[la, B[lc, ld]] × CD[lb, B[ud, uc]]}
% // Tsimplify
```

```
Out[*]= {Aab ∂aBcd ∂bBcd, Aab ∇aBcd ∇bBcd, Aab ∇aBcd ∇bBdc}
```

```
Out[*]= {Aab ∂aBcd ∂bBcd, 0, 0}
```

```
In[*]:= {A[la, lb] × CD[le, ua, B[lc, ld]] × CD[lf, ub, B[uc, ud]],
        A[la, lb] × CD[le, ua, B[lc, ld]] × CD[ue, ub, B[uc, ud]]}
% // Tsimplify
```

```
Out[*]= {Aab ∇e∇aBcd ∇f∇bBcd, Aab ∇c∇aBde ∇c∇bBde}
```

```
Out[*]= {Aab ∇e∇aBcd ∇f∇bBcd, 0}
```

$$A^{[ab]} \partial_a g_{pq} \partial_b g^{pq} = 0$$

```
In[*]:= expr = {A[ua, ub] × BD[la, Metricg[lc, ld]] × BD[lb, Metricg[uc, ud]], A[ua, ub] ×
        CD[le, BD[la, Metricg[lc, ld]]] × CD[ue, BD[lb, Metricg[uc, ud]]]}
expr // Tsimplify (* These should be zero! *)
```

```
Out[*]= {Aab ∂agcd ∂bgcd, Aab ∇c∂agde ∇c∂bgde}
```

```
Out[*]= {Aab ∂agcd ∂bgdc, Aab ∇c∂agde ∇c∂bgde}
```

```
In[*]:= Tsimplify[expr /. BDinvRule[]]
```

```
Out[*]= {0, 0}
```

```
In[*]:= {A[ua, ub] × CD[le, BD[la, Metricg[lc, ld]]] × CD[lf, BD[lb, Metricg[uc, ud]]],
        A[ua, ub] × BD[le, BD[la, Metricg[lc, ld]]] × BD[ue, BD[lb, Metricg[uc, ud]]]}
% // Tsimplify
```

```
Out[*]= {Aab ∇e∂agcd ∇f∂bgcd, Aab ∂c∂agde ∂c∂bgde}
```

```
Out[*]= {Aab ∇e∂agcd ∇f∂bgdc, Aab ∂a∂cgde ∂c∂bgde}
```

Multi-Terms

```
In[*]:= {B[la, ua] + B[ua, la], CD[la, lb, B[ua, ub]] + CD[lc, ld, B[uc, ud]]}
% // Tsimplify
```

```
Out[*]=
```

$$\{B_a^a + B_a^a, 2 \nabla_a \nabla_b B^{ab}\}$$

```
Out[*]=
```

$$\{2 B_a^a, 2 \nabla_a \nabla_b B^{ab}\}$$

```
In[*]:= {B[la, lb] × B[ua, ub] + B[lc, ld] × B[uc, ud],
B[la, lb] × B[ub, ua] + B[la, lb] × B[ua, ub],
B[lc, la] × B[uc, lb] + B[lc, lb] × B[uc, la]}
% // Tsimplify
```

```
Out[*]=
```

$$\{2 B_{ab} B^{ab}, B_{ab} B^{ab} + B_{ab} B^{ba}, B_{cb} B^c_a + B_{ca} B^c_b\}$$

```
Out[*]=
```

$$\{2 B_{ab} B^{ab}, B_{ab} B^{ab} + B_{ab} B^{ba}, 2 B_{ca} B^c_b\}$$

```
In[*]:= {S[la, uc] × S[lc, lb] + S[le, la] × S[lb, ue],
RiemannCD[la, lb, ld, le] × S[ua, ue] + RiemannCD[lb, le, lc, ld] × S[ue, uc]}
% // Tsimplify
```

```
Out[*]=
```

$$\{S_b^c S_{ca} + S_a^c S_{cb}, R_{abdc} S^{ac} + R_{bacd} S^{ac}\}$$

```
Out[*]=
```

$$\{2 S_{ac} S_b^c, -2 R_{badc} S^{ac}\}$$

```
In[*]:= RiemannCD[la, lb, lc, ld] × RiemannCD[le, lf, uc, ud] ×
RiemannCD[ua, ub, ue, uf] + RiemannCD[lc, ld, ug, uh] ×
RiemannCD[le, lf, lg, lh] × RiemannCD[uc, ud, ue, uf]
% // Tsimplify
```

```
Out[*]=
```

$$R_{ab}^{cd} R_{efcd} R^{abef} + R_{abcd} R_{ef}^{cd} R^{abef}$$

```
Out[*]=
```

$$2 R_{abcd} R_{ef}^{ab} R^{cdef}$$

```
In[*]:= CD[ua, S[lb, la]] × CD[ub, f[]] + CD[ub, S[lb, la]] × CD[ua, f[]]
% // Tsimplify
```

```
Out[*]=
```

$$\nabla^a S_{ba} \nabla^b f + \nabla^a f \nabla^b S_{ba}$$

```
Out[*]=
```

$$2 \nabla_a f \nabla_b S^{ab}$$

More examples

```
In[*]:= Tscalar[A[uc, ud] × A[lc, ld]]
% // Tsimply
```

```
Out[*]=
```

$$(A_{cd} A^{cd})$$

```
Out[*]=
```

$$(A_{cd} A^{cd})$$

```
In[*]:= expr = Tscalar[A[uc, ud] × A[lc, ld]]^2 Tscalar[A[la, lb] × A[ub, ua]]
Tsimply[expr]
Tsimply[expr, HeadQs → {ObjectQ}] (* Messages in ver 14.2. Why? *)
```

```
Out[*]=
```

$$(A_{ab} A^{ba}) (A_{cd} A^{cd})^2$$

```
Out[*]=
```

$$(A_{ab} A^{ba}) (A_{cd} A^{cd})^2$$

... **Symbol:** The string "xObject [Tscalar [A[la, lb] A[ua, ub]], {A[la, lb] A[ua, ub]}, {{}, {}, {}}, {}]" cannot be used for a symbol name. A symbol name must start with a letter followed by letters and numbers. [i](#)

... **Symbol:** The string "PowerTscalar [A[la, lb] A[ua, ub]^2]" cannot be used for a symbol name. A symbol name must start with a letter followed by letters and numbers. [i](#)

... **Symbol:** The string "Symbol [xObject [Tscalar [A[la, lb] A[ua, ub]], {A[la, lb] A[ua, ub]}, {{}, {}, {}}, {}]]Symbol [PowerTscalar [A[la, lb] A[ua, ub]^2]" cannot be used for a symbol name. A symbol name must start with a letter followed by letters and numbers. [i](#)

... **General:** Further output of Symbol::symname will be suppressed during this calculation. [i](#)

```
Out[*]=
```

$$- (A_{ab} A^{ab})^3$$

With repeated indices:

```
In[*]:= Tdefine[H, "3-"]
```

```
In[*]:= H[ub, ua, uc]
% // Tsimply
```

```
Out[*]=
```

$$H^{bac}$$

```
Out[*]=
```

$$-H^{abc}$$

<code>In[]:=</code>	<code>H[2, 1, 3]</code> <code>% // Tsimplify</code>
<code>Out[]:=</code>	H^{213}
<code>Out[]:=</code>	$-H^{123}$
<code>In[]:=</code>	<code>H[1, 1, 3]</code> <code>% // Tsimplify</code>
<code>Out[]:=</code>	H^{113}
<code>Out[]:=</code>	0
<code>In[]:=</code>	<code>H[1, 3, 1]</code> <code>% // Tsimplify</code>
<code>Out[]:=</code>	H^{131}
<code>Out[]:=</code>	0